

A BEGINNER'S COURSE BOOK

Let's Code with PictoBlox!

An 11-Week Journey into Block Coding, for Young Coders Ages 5-11



One Virtual Class per Week · Hands-On Practice · From First Script to First Game

Contents

Wk 1 Meet PictoBlox!

Wk 2 Motion Explorers - Move, Turn, and Go

Wk 3 Looks & Expression

Wk 4 Sound & Music

Wk 5 Events - Many Ways to Start a Script

Wk 6 Loops - Repeat vs Forever

Wk 7 Making Decisions - If and If/Else

Wk 8 Variables & Operators - Keeping Score

Wk 9 Lists & My Blocks - Staying Organized

Wk 10 Build a Game - Catch the Falling Object

Wk 11 Showcase & What's Next

Welcome, Teacher!

This course book turns block coding into an 11-week virtual class for young coders aged 5-11. Each week is written as a single, ready-to-teach lesson: a hook to open the session, the blocks being introduced, a picture of them snapped together, a "why this block and not that one" comparison that builds real understanding (not just copying), a live teaching script, a take-home practice activity, and a reflection question to close.

How to run each session

- Meet once a week virtually (about 45-60 minutes works well for this age range).
- Open with the Hook - read it aloud, or in your own words, to set the scene.
- Walk through "Meet the blocks" together, live in PictoBlox, sharing your screen.
- Use the "Snap them together" diagram to show exactly how the script should look before students try it.
- Talk through the "Why this and not that" box - this is where real understanding is built.
- Let students try the "Try it yourself" activity during the session or as practice before next week.
- Close with the reflection question - even a one-sentence answer in the chat helps lock in the idea.

A note on the block colors

PictoBlox always colors its blocks by family. Once students recognize the colors, they can find blocks much faster - gold for Events, blue for Motion, purple for Looks, magenta for Sound, orange for Control, cyan for Sensing, green for Operators, and reddish-orange for Variables. Point this out early and often.

WEEK 1

Meet PictoBlox!

Welcome, coders! Today you get to meet a brand-new friend: a computer program called PictoBlox. It's like a box full of LEGO bricks, except every brick is an instruction a sprite can follow. Snap two bricks together and something happens on the screen!

What we will learn today

- Know what a sprite, a stage, and a script are
- Understand that a program is just instructions in order
- Build your very first script and press the green flag

Meet the blocks

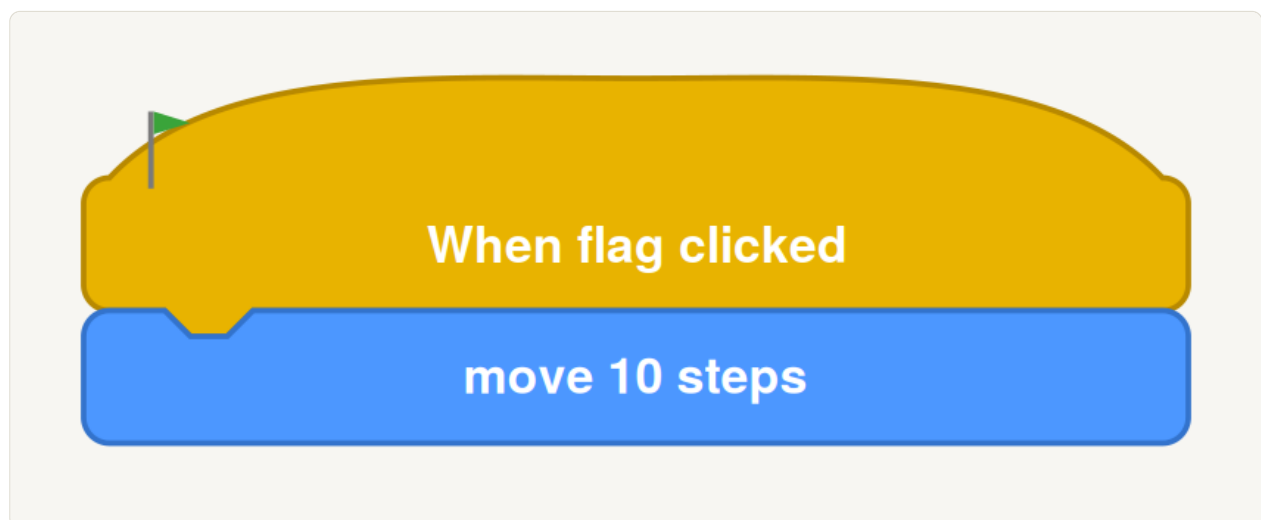
When flag clicked

This is a HAT block. It sits at the top of a script and waits. Nothing happens until someone clicks the green flag.

move 10 steps

This is a STACK block. It tells the sprite to walk forward a little bit.

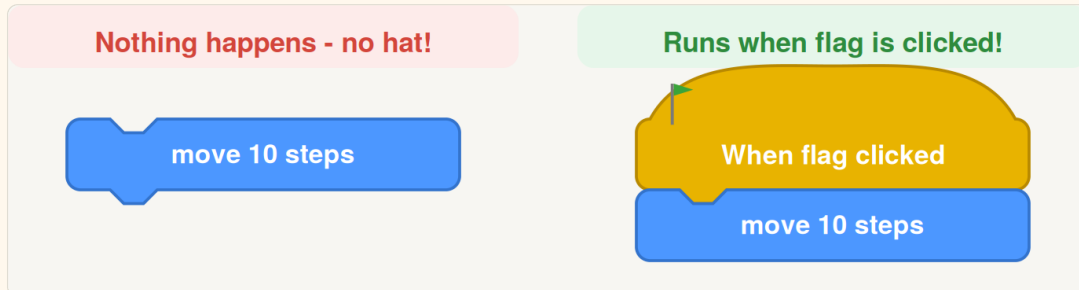
Snap them together!



Your first script: a hat block on top, one action block underneath.

Why do we need the hat block at all?

A stack block by itself is like a light switch with no wire to the wall - it has an instruction, but nothing to trigger it. The hat block is the wire. Without it, PictoBlox has no idea WHEN to run your block, so it just sits there, unused.



Left: a lonely block with no hat - it never runs. Right: the same block under a hat - it runs!

Teacher walkthrough (live session)

1. Open PictoBlox and look at the cat sprite sitting on the white Stage.
2. Find the Events palette (it's gold/yellow) and drag out 'when flag clicked'.
3. Find the Motion palette (it's blue) and drag out 'move 10 steps'. Snap it under the hat - you'll feel/see it click into place.
4. Click the green flag above the Stage. Watch your cat move!
5. Click flag a few more times. Notice it always starts from where it stopped, not from zero - we'll fix that next week!

Try it yourself

Make your sprite move 10 steps, then add a second 'move 10 steps' block under the first. Click the flag - does it move further? Try changing the number 10 to 50. What happens?

Think about it

If you deleted the hat block and just left 'move 10 steps' sitting alone in the coding area, what would happen if you clicked the green flag? Why?

WEEK 2

Motion Explorers - Move, Turn, and Go

Last week our sprite took its first steps. This week, we're going to teach it to walk in shapes, spin around, and always know exactly where to start from.

What we will learn today

- Use 'go to x/y' to always start in the same place
- Tell the difference between move, turn, and point in direction
- Combine several motion blocks into a longer script

Meet the blocks

go to x: 0 y: 0

Teleports the sprite to the exact center of the Stage. Great for a 'reset' at the start.

turn 90 degrees

Spins the sprite a certain amount RELATIVE to the way it's already facing.

point in direction 90

Rocket-turns the sprite to face one EXACT direction, no matter which way it faced before.

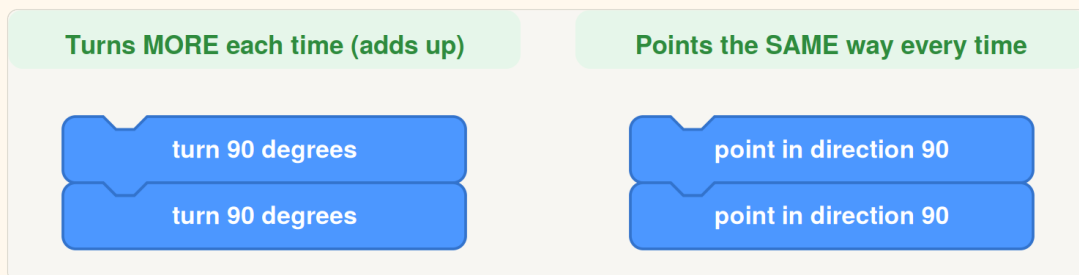
Snap them together!



Start at (0,0), move forward, then turn - a simple exploring pattern.

🤔 turn 90 degrees vs point in direction 90 - why does it matter which one I pick?

'turn' is like telling a friend 'spin a quarter turn from wherever you're facing now' - do it four times in a row and they've gone all the way around. 'point in direction' is like telling them 'face East, no matter what' - say it four times and they're still just facing East. Use 'turn' when you want to keep adding rotation (like drawing a square). Use 'point in direction' when you want a guaranteed, exact heading.



Left: turn adds up each time. Right: point in direction always snaps to the same heading.



Teacher walkthrough (live session)

1. Start every script this week with 'when flag clicked' then 'go to x: 0 y: 0' - this is now our warm-up habit.
2. Add 'move 50 steps' then 'turn 90 degrees'. Click the flag several times in a row and watch the direction change each time.
3. Now swap 'turn 90 degrees' for 'point in direction 90'. Click the flag several times - what's different?
4. Challenge: can you make the sprite walk in an 'L' shape using go to, move, and turn?



Try it yourself

Build a script that makes your sprite walk forward, turn, walk forward again, and turn again - four times in a row. What shape does it draw if you add a pen down block (we'll learn Pen later, but you can guess first)?



Think about it

Why do we start almost every script with 'go to x: 0 y: 0'? What might go wrong if we skipped it?

WEEK 3

Looks & Expression

A sprite that can only move is a little boring. This week we give it a voice, a face, and a costume change - so it can tell a story, not just walk around.

What we will learn today

- Make a sprite talk with 'say' and 'think'
- Understand show vs hide
- Change costumes to animate a sprite

Meet the blocks

show / hide

Controls whether the sprite is visible on the Stage at all.

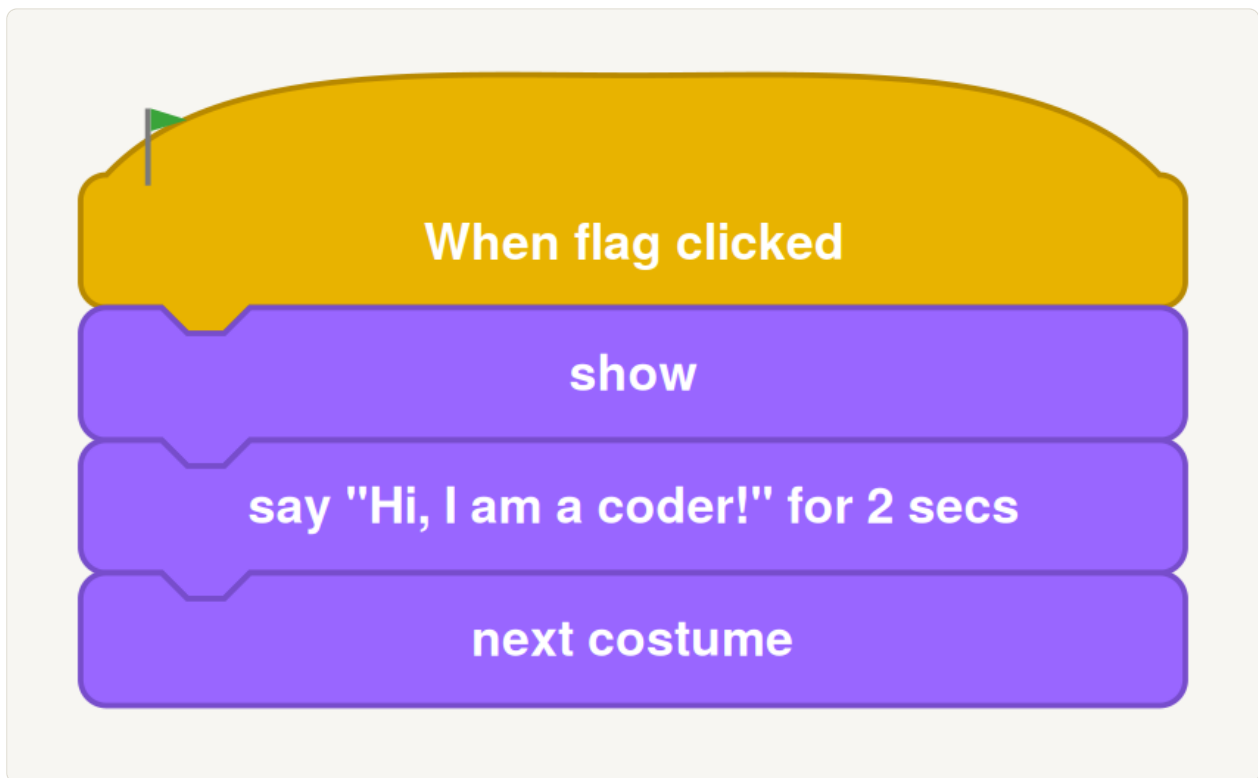
say ... for 2 seconds

Pops up a speech bubble with your sprite's words, for a set amount of time.

next costume

Switches to the next picture in the sprite's costume list - the trick behind animation.

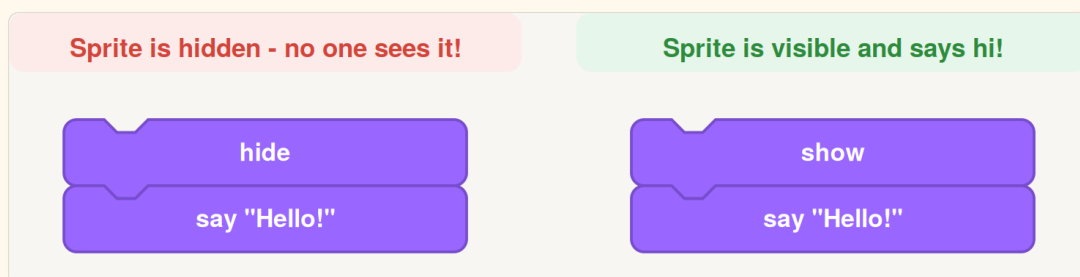
Snap them together!



Show the sprite, have it greet the class, then switch its costume.

Why does order matter here - show BEFORE say?

PictoBlox runs your blocks top to bottom, one at a time, like reading a recipe. If the sprite starts hidden and you put 'say' before 'show', the words appear over an invisible sprite - nobody can tell who is talking! Block order is part of the plan, not just decoration.



Left: sprite is still hidden when it talks - no one can see who's speaking. Right: shown first, so the greeting makes sense.

Teacher walkthrough (live session)

1. Drag out 'when flag clicked', then 'show', then 'say Hi I am a coder! for 2 seconds'.

2. Click the flag. Then try moving 'show' to the very bottom of the script and click flag again - talk about what changes.
3. Add 'next costume' after the say block, and click flag a few times fast - can you see the costume change?
4. Try 'think' instead of 'say' - how is a thought bubble different from a speech bubble?

Try it yourself

Give your sprite a two-line conversation with itself using two 'say for 2 seconds' blocks in a row. Then add a costume change in between the two lines.

Think about it

What is one example from real life where doing things in the wrong order causes a problem (like putting on shoes before socks)? How is that like block order in PictoBlox?

Sound & Music

Today our sprites get their voice back in a different way - through sound! We'll learn that PictoBlox gives us two very different ways to play a sound, and picking the right one changes everything.

What we will learn today

- Play a sound effect when something happens
- Understand play sound vs play sound until done
- Combine sound with motion for feedback

Meet the blocks

play sound Pop

Starts the sound playing, then IMMEDIATELY moves on to the next block - it doesn't wait.

play sound Pop until done

Starts the sound AND waits for it to finish before moving to the next block.

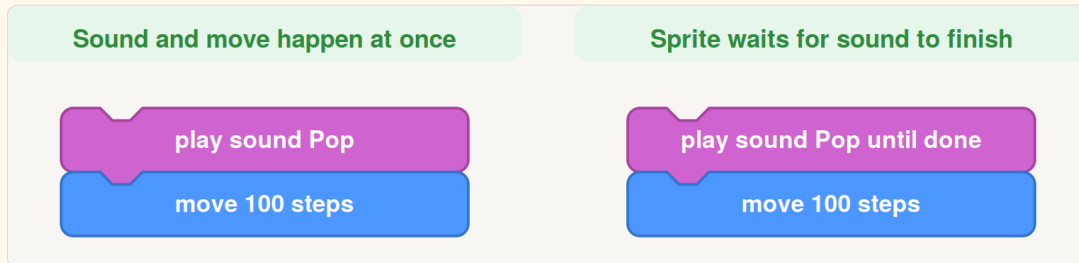
Snap them together!



The sprite waits for the pop sound to finish before it moves.

🤔 play sound vs play sound until done - which one do I need?

Think of a walkie-talkie. 'play sound' is like saying 'go!' and running off while the message is still playing in the background - useful for background music or footsteps while the sprite keeps moving. 'play sound until done' is like waiting for someone to finish a whole sentence before you respond - perfect when the sound IS the important thing, like a countdown 'ready, set, go!'



Left: sound and movement happen together. Right: sprite waits, then moves after the sound ends.

👩🏫 Teacher walkthrough (live session)

1. Open the Sound tab and pick (or record!) a short sound for your sprite.
2. Build: when flag clicked -> play sound Pop -> move 100 steps. Notice the sprite moves right away.
3. Now swap in 'play sound Pop until done'. Click flag again - what's different about the timing?
4. Try attaching a sound to a key-press event instead of the flag, so pressing space makes a noise.

✎ Try it yourself

Add a sound effect that plays every time your sprite bumps the edge of the Stage (hint: you'll need an if block - a sneak peek at Week 7!). For now, just trigger it from a key press.

💭 Think about it

Can you think of a moment in a game where you'd want the game to WAIT for a sound (like a 'level complete' jingle) instead of rushing ahead?

Events - Many Ways to Start a Script

So far every script has started with the green flag. But real programs react to lots of different things - a key press, a click, even a message from another script! Today we unlock all of them.

What we will learn today

- Start scripts with a key press or a sprite click
- Use broadcast and 'when I receive' to make scripts talk to each other
- Understand that several scripts can run at the same time

Meet the blocks

when [space] key pressed

Starts a script the instant a chosen key is pressed.

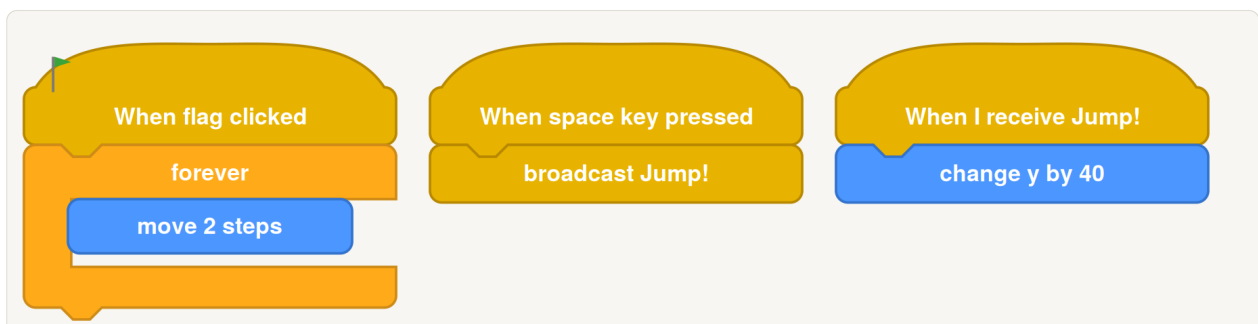
broadcast Jump!

Sends an invisible signal to every script in the project - like shouting a message.

when I receive Jump!

A hat block that wakes up **ONLY** when it hears its matching broadcast.

Snap them together!



Three separate scripts running side by side - each with its own hat block.

🤔 Why not just put everything in one giant script?

Imagine one student trying to answer the phone, cook dinner, and walk the dog all by following ONE very long list of instructions - by the time they get to 'answer the phone' on line 40, the caller already

hung up! Broadcasts let you split a big program into small scripts that react instantly, each one is only responsible for its own job.



Teacher walkthrough (live session)

1. Build a forever-move script that starts on the green flag (we'll look closely at forever next week).
2. In a NEW script, add 'when space key pressed' -> 'broadcast Jump!'
3. In a THIRD script, add 'when I receive Jump!' -> 'change y by 40'.
4. Click the flag, then tap space while the sprite is moving. Two scripts just cooperated!



Try it yourself

Add a fourth script that plays a sound 'when this sprite clicked'. Click your sprite while the other scripts are running - do they all work at once?



Think about it

What is the difference between 'broadcast' and 'broadcast and wait'? When might waiting matter (hint: think about our Week 4 sound lesson)?

Loops - Repeat vs Forever

Typing the same block ten times would take forever (pun intended!). Today we learn PictoBlox's two favorite loop blocks, and - most importantly - when to use which one.

What we will learn today

- Use 'repeat' for a loop that runs a set number of times
- Use 'forever' for a loop that never stops on its own
- Choose the right loop for the right job

Meet the blocks

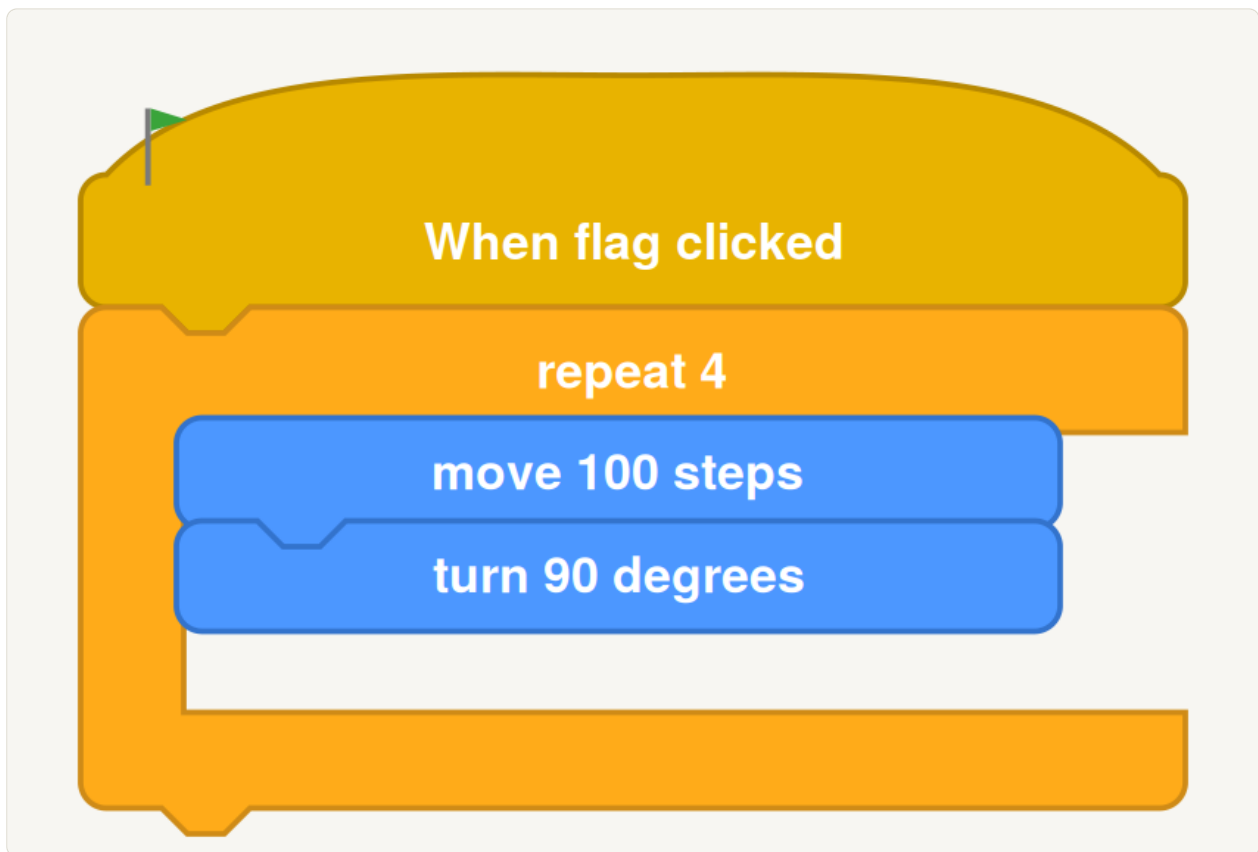
repeat 4

A C-shaped block that runs whatever is snapped inside it a fixed number of times, then continues on.

forever

A C-shaped block that runs whatever is inside it again and again, forever - it never lets go.

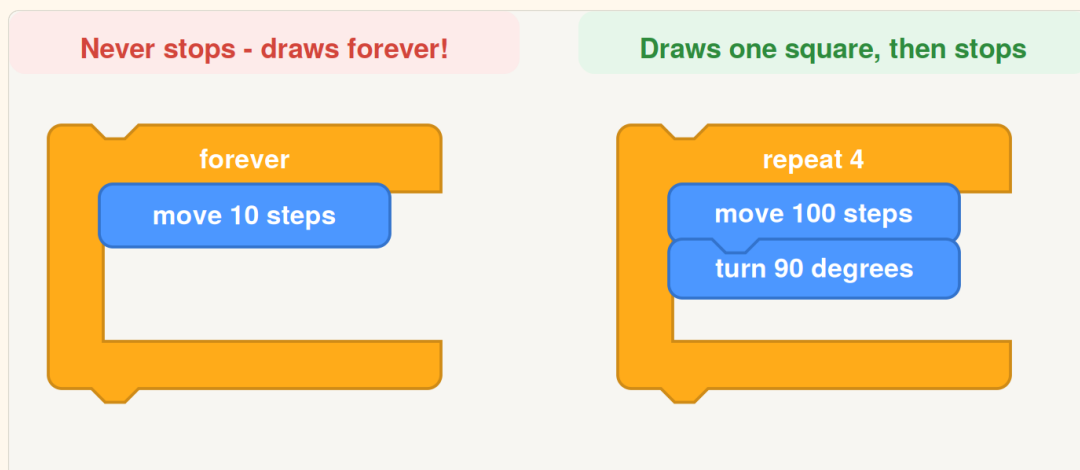
Snap them together!



repeat 4 draws exactly one square: move, turn, move, turn... then stops.

🤔 forever vs repeat 4 - why does picking the wrong one break your project?

'forever' is perfect for things that should keep happening the whole game, like a sprite always checking if it touches an enemy. But if you use 'forever' to draw a square, PictoBlox keeps drawing that square over and over, forever, and your script never moves on to the next block below it. 'repeat 4' does the job exactly four times and then politely lets the rest of your script run.



Left: forever never lets the square-drawing finish. Right: repeat 4 finishes and moves on.



Teacher walkthrough (live session)

1. Build: when flag clicked -> repeat 4 [move 100 steps, turn 90 degrees]. Click the flag - a square appears (add a Pen 'pen down' block if you want to see the trail).
2. Now change 'repeat 4' to 'forever' and click flag again. What happens? How do you stop it (hint: the red stop sign)?
3. Try 'repeat 10' with a smaller move distance - what shape or pattern do you get?



Try it yourself

Use 'forever' to make your sprite check the edge and bounce, non-stop, the whole time the project runs. Then use 'repeat' to make it spin exactly one full circle (hint: 360 divided by your turn amount).



Think about it

Name one thing in a video game that should use 'forever' and one thing that should use 'repeat'. Why did you choose each one?

Making Decisions - If and If/Else

Real life is full of decisions: if it's raining, take an umbrella. Today our sprites learn to make decisions too, using sensing blocks to check the world around them.

What we will learn today

- Use sensing blocks like 'touching edge?' inside a condition
- Use 'if...then' for a decision with one outcome
- Know when 'if/else' is the better choice - and when it causes bugs

Meet the blocks

touching edge?

A hexagon-shaped BOOLEAN block - it reports true or false, and plugs into the notch of an if block.

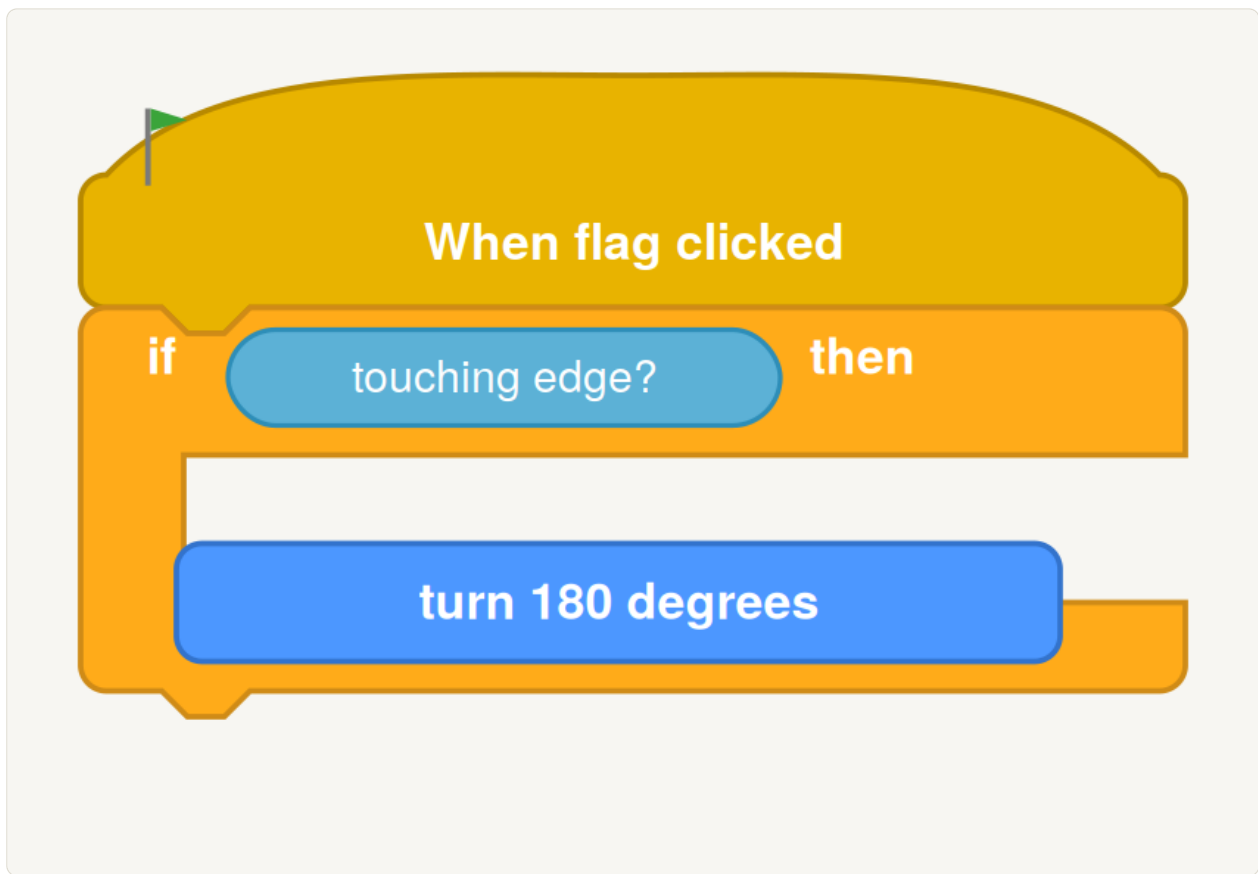
if <> then

Runs the blocks inside ONLY when the boolean condition is true. If false, it just skips them.

if <> then / else

Runs one set of blocks if true, and a DIFFERENT set if false - always one or the other.

Snap them together!



if touching edge? then turn 180 degrees - a simple bounce.

🤔 if vs if/else - why can 'else' accidentally cause a bug?

'else' ALWAYS runs when the condition is false - every single time PictoBlox checks, dozens of times per second. If your 'else' branch does something like 'set score to 0', you've just built a script that resets the score constantly, any moment the sprite ISN'T touching the edge! Plain 'if' is safer whenever you only care about the true case and want nothing to happen otherwise.

else runs EVERY other frame - bug!



Only bounces when it touches edge



Left: the else branch quietly resets the score almost every frame - a hidden bug. Right: a plain if only bounces exactly when needed.



Teacher walkthrough (live session)

1. Drag out the sensing hexagon 'touching edge?' - notice its shape matches the notch in the if block, like a puzzle piece.
2. Build: when flag clicked -> forever -> if touching edge? then turn 180 degrees, move 10 steps.
3. Click flag and watch your sprite bounce off the walls forever.
4. Now build the 'buggy' else example from the diagram and talk about WHY the score keeps resetting.



Try it yourself

Make a sprite that says 'Ouch!' only when it touches the edge, and stays quiet the rest of the time. Should you use if, or if/else? Try both and see which one behaves the way you want.



Think about it

Can you think of a real decision that only needs 'if' (with nothing happening otherwise), and one that really needs 'if/else' (something should always happen either way)?

Variables & Operators - Keeping Score

Games need to remember things - like a score! This week we create our own variable, a little labeled box that stores a number, and we learn two very different ways to fill it.

What we will learn today

- Create a variable and understand it stores one value at a time
- Use set vs change and know exactly when to use each
- Combine a variable with an if block to build a scoring system

Meet the blocks

set score to 0

Puts an EXACT value into the variable, wiping out whatever was there before.

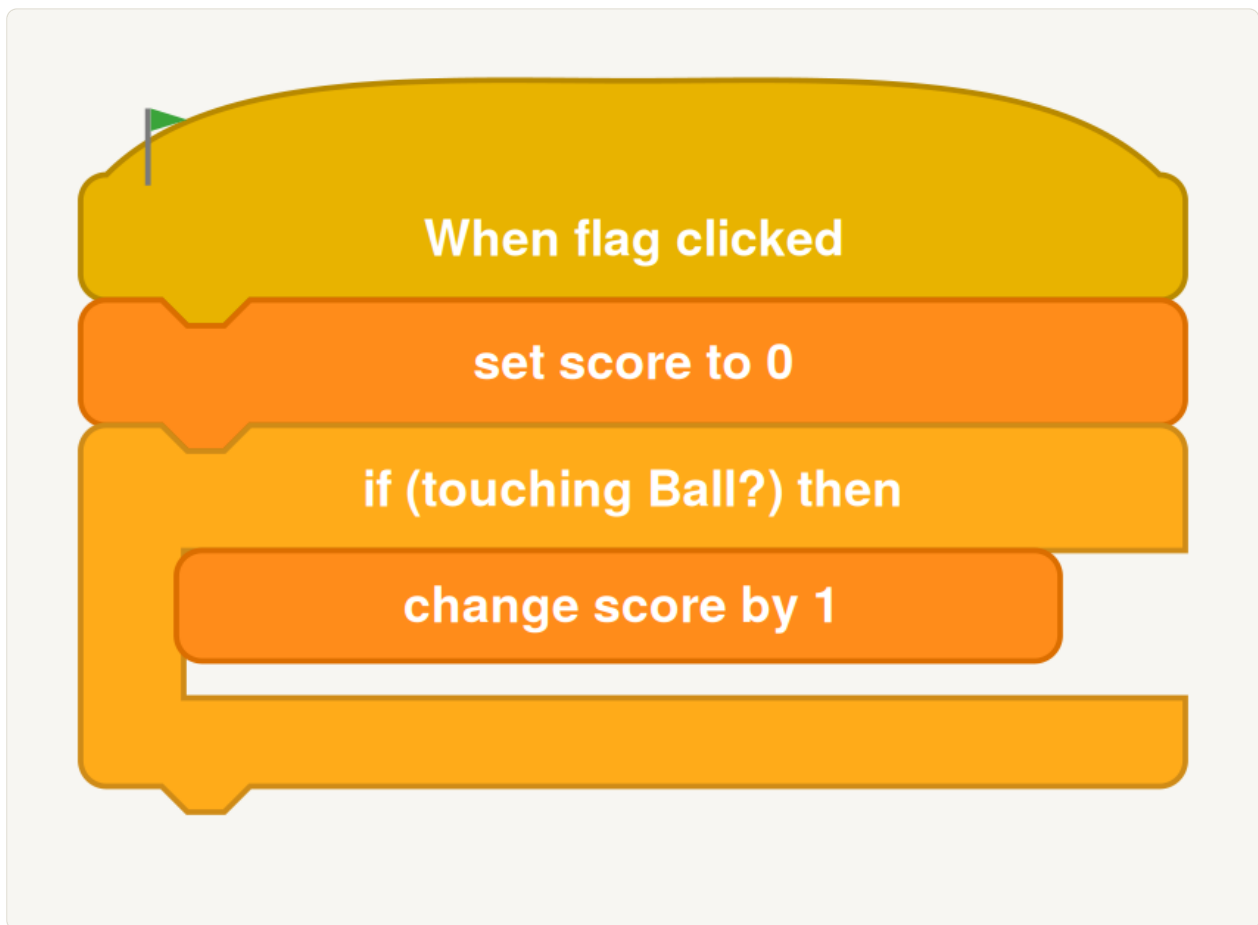
change score by 1

ADDS to whatever value is already there - it depends on the old number.

score > 10

A green hexagon reporter that compares two values and reports true or false.

Snap them together!



set score to 0 once, then change score by 1 every time the sprite touches the ball.

🤔 set vs change - why does the ORDER of my blocks matter so much here?

'set' is like writing a fresh number on a whiteboard, erasing whatever was there. 'change' is like adding to the number that's already written. If you use 'change score by 0' at the start of your game instead of 'set score to 0', the old score from your LAST game is still sitting there, and your new game starts with a leftover number instead of a fair, fresh zero!



Left: change by 0 does nothing to the old leftover score. Right: set to 0 always gives a fresh, fair start.



Teacher walkthrough (live session)

1. Go to the Variables palette and click 'Make a Variable'. Name it 'score'.
2. Build: when flag clicked -> set score to 0 -> forever -> if touching Ball? then change score by 1.
3. Play a few rounds and watch the little score display in the top corner of the Stage change.
4. Try using the operator block 'score > 10' inside an if, to make your sprite say 'You win!' once the score is high enough.



Try it yourself

Build a mini game where touching a 'Ball' sprite adds 1 to score, and when score is greater than 5, your sprite says 'You win!'.



Think about it

Why do we always put 'set score to 0' right after the green flag, and never inside the forever loop?

Lists & My Blocks - Staying Organized

A variable can only hold ONE thing at a time - but what if we need to remember a whole shopping list, or a whole leaderboard? And what happens when the same three blocks show up again and again in your project? Today we solve both problems.

What we will learn today

- Create a list and add items to it
- Understand why repeating blocks gets messy
- Build a custom My Block to reuse a set of actions with one clean block

Meet the blocks

add "apple" to Shopping List

Adds one more item onto the end of a list - a list can hold many values, unlike a single variable.

jump

A custom block YOU design. Define it once with a set of actions inside, then reuse it anywhere with a single snap.

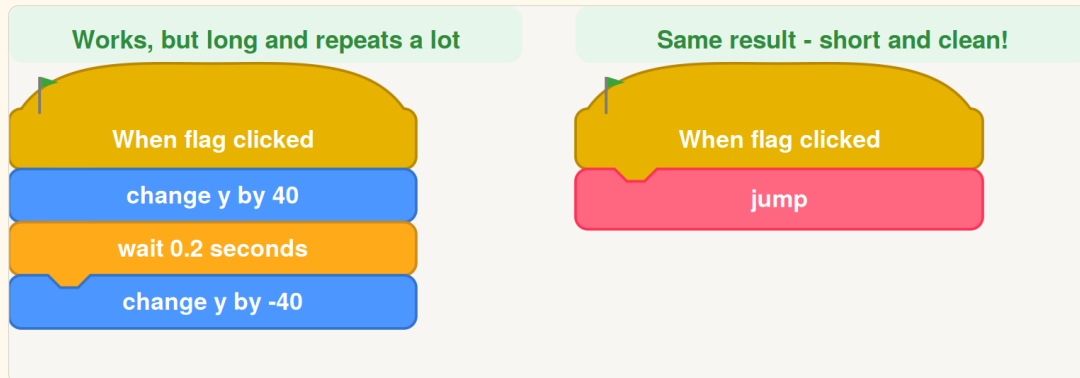
Snap them together!



Two items being added, one after another, to a Shopping List.

Repeating the same three blocks vs making a My Block - why bother?

Copy-pasting the same three blocks every time your sprite needs to jump works, but it makes your project long, and if you ever want to change how a jump feels, you'd have to fix it in every single copy. A My Block is like giving that group of actions its own name - 'jump' - so you write it once and reuse it as many times as you like. If you improve it later, you only change it in one place.



Left: the same 3 blocks copied out in full. Right: one clean custom 'jump' block doing the same job.

Teacher walkthrough (live session)

1. Go to Variables, click 'Make a List', name it Shopping List, and use 'add' to put a few items in.
2. Go to My Blocks, click 'Make a Block', and name it 'jump'.
3. Inside the new 'define jump' hat, add: change y by 40, wait 0.2 seconds, change y by -40.
4. In your main script, just snap in the pink 'jump' block whenever you want your sprite to jump - one block instead of three!

Try it yourself

Make a My Block called 'greet' that shows the sprite, says hello, and plays a sound - then use it in two different scripts (like on flag click AND on sprite click).

Think about it

If you had 10 places in your project where a sprite needed to jump, how many blocks would you need WITHOUT a My Block, versus WITH one?

Build a Game - Catch the Falling Object

You've learned motion, looks, sound, events, loops, decisions, and variables. Today we combine every single one of them into your first real game!

What we will learn today

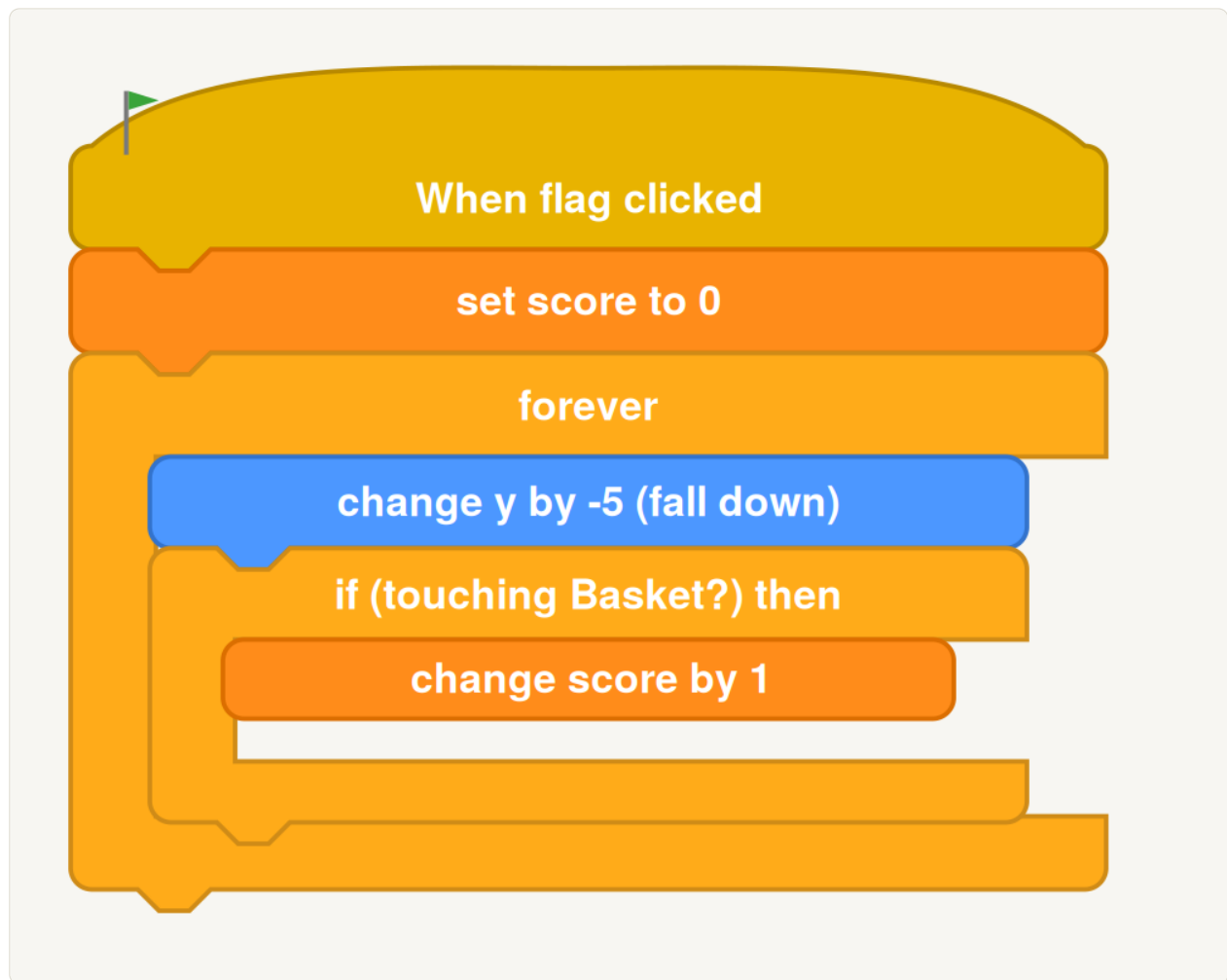
- Plan a game before building it
- Combine loops, conditions, and variables in one script
- Debug a script when something doesn't behave as expected

Meet the blocks

forever + if, nested together

A loop that keeps checking a condition again and again, every single frame - the heart of most games.

Snap them together!



The whole game script: reset score, then forever make the object fall and check if it's caught.

Why does 'set score to 0' have to live OUTSIDE the forever loop?

This is the same idea from Week 8, now inside a real game: if 'set score to 0' were accidentally placed **INSIDE** the forever loop, it would wipe your score back to zero dozens of times every second, and you could never win. Planning **WHERE** each block goes - not just **WHICH** block - is what separates a working game from a broken one.

Teacher walkthrough (live session)

1. Sketch your game on paper first: What falls? What catches it? What happens when they touch?
2. Build: when flag clicked -> set score to 0 -> forever -> change y by -5 (falling) -> if touching Basket? then change score by 1.

3. Add: when the falling object reaches the bottom of the Stage, send it back to the top (hint: 'if y < -170 then go to x: pick random, y: 170').
4. Playtest it yourself. If something looks wrong, read the script out loud, block by block, like a recipe - this is real debugging!

Try it yourself

Finish your Catch game and add ONE improvement of your own choosing: a sound on catch, a 'Game Over' message after 30 seconds, or a second falling object.

Think about it

What was the trickiest bug you ran into today, and how did you figure out what was wrong?

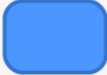
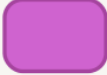
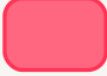
Showcase & What's Next

You started eleven weeks ago not knowing what a sprite was. Today, you're going to show the class the game YOU built, explain your choices like a real programmer, and take a peek at what comes next.

What we will learn today

- Present a finished project and explain how it works
- Reflect on the concepts learned across the course
- Preview physical computing and robotics as a next step

Snap them together!

-  Events - starts a script
-  Motion - moves the sprite
-  Looks - changes appearance
-  Sound - plays audio
-  Control - loops & decisions
-  Sensing - detects the world
-  Operators - math & compare
-  Variables - stores data
-  My Blocks - custom actions

Every block family we met this term, and the job each color does.

Why did we learn things in THIS order?

Notice the path we walked: instructions, then motion and looks, then events, then loops, then decisions, then variables, then organizing with lists and My Blocks - and finally, we combined all of it into one real game. Each new idea leaned on the one before it. That's exactly how professional programmers grow too: one solid idea at a time, then combine them.

Teacher walkthrough (live session)

1. Each student shares their screen and clicks the green flag on their finished project.
2. The presenter explains: 'This block does X, and I used if/else here because...' - practicing real explanation, not just showing.
3. Classmates ask one question each: 'What was hard?' or 'What would you add next?'
4. Wrap-up: a short look at what's next - Quarky boards, LEDs, and sensors, where the SAME blocks you learned start controlling real hardware.

Try it yourself

Write down the three PictoBlox concepts you feel most confident about, and the one you want more practice with.

Think about it

If you could add ONE new feature to your game using something we have NOT learned yet, what would it be, and what do you think you'd need to learn to build it?